Object Detection Pipeline on Kubernetes

CSC 581 – Cloud Computing | Fall 2026

Logan Yates

West Chester University of Pennsylvania

September 2026

Chapter 1: Vision

1.1 Overview

For every second, billions of cameras around the world capture images that no human will ever review. Security feeds go unwatched. Medical scans wait in queues. Retail shelves go unaudited. The bottleneck is not the hardware; it is the intelligence at scale. Object detection, the ability to pinpoint and name every meaningful element within an image, has become one of the most transformative capabilities in modern software. It powers the defect spotting on factory assembly lines, the shoplifting alerts in retail stores, and the crowd-density monitoring in public spaces.

Deploying object detection in production is not simply a matter of running a model on a laptop. Real-world systems must handle scale under load, version their components independently, and minimize the chance that a result is silently lost. These are infrastructure problems and Kubernetes is the industry's answer to them.

This project delivers a fully containerized, cloud-native system that accepts a raw image from a user, preprocesses it into the precise data block format required by a neural network, runs it through a pre-trained MobileNet-SSD model capable of detecting 90 distinct object classes, and stores the original image, the processed data block, and a structured JSON result containing every detected object's label, confidence score, and bounding box. In Milestone 2 this storage is ephemeral; persistent storage is planned for Milestone 3. The system is designed as a set of loosely coupled microservices, each with a separate Kubernetes Pod communicating over private, cluster-internal ClusterIP Services, with exactly one external entry point exposed to the world.

The architectural choices are deliberate. A single NodePort gateway keeps the attack surface minimal. Pinned container image versions make every deployment reproducible. In Milestone 2, the storage service uses an emptyDir volume with a structured directory layout; in Milestone 3 this will be replaced by a shared PersistentVolumeClaim (PVC) so that artifacts are preserved across Pod restarts and crashes mid-pipeline. And the inference Deployment, the compute heart of the system, is the designated horizontal scale target: add more replicas and throughput increases, without touching any other component.

MobileNet-SSD was selected deliberately. At roughly 20 MB, it is small enough to load in seconds, fast enough to run on CPU without a GPU, and accurate enough to detect real objects in real images.

1.2 Architecture

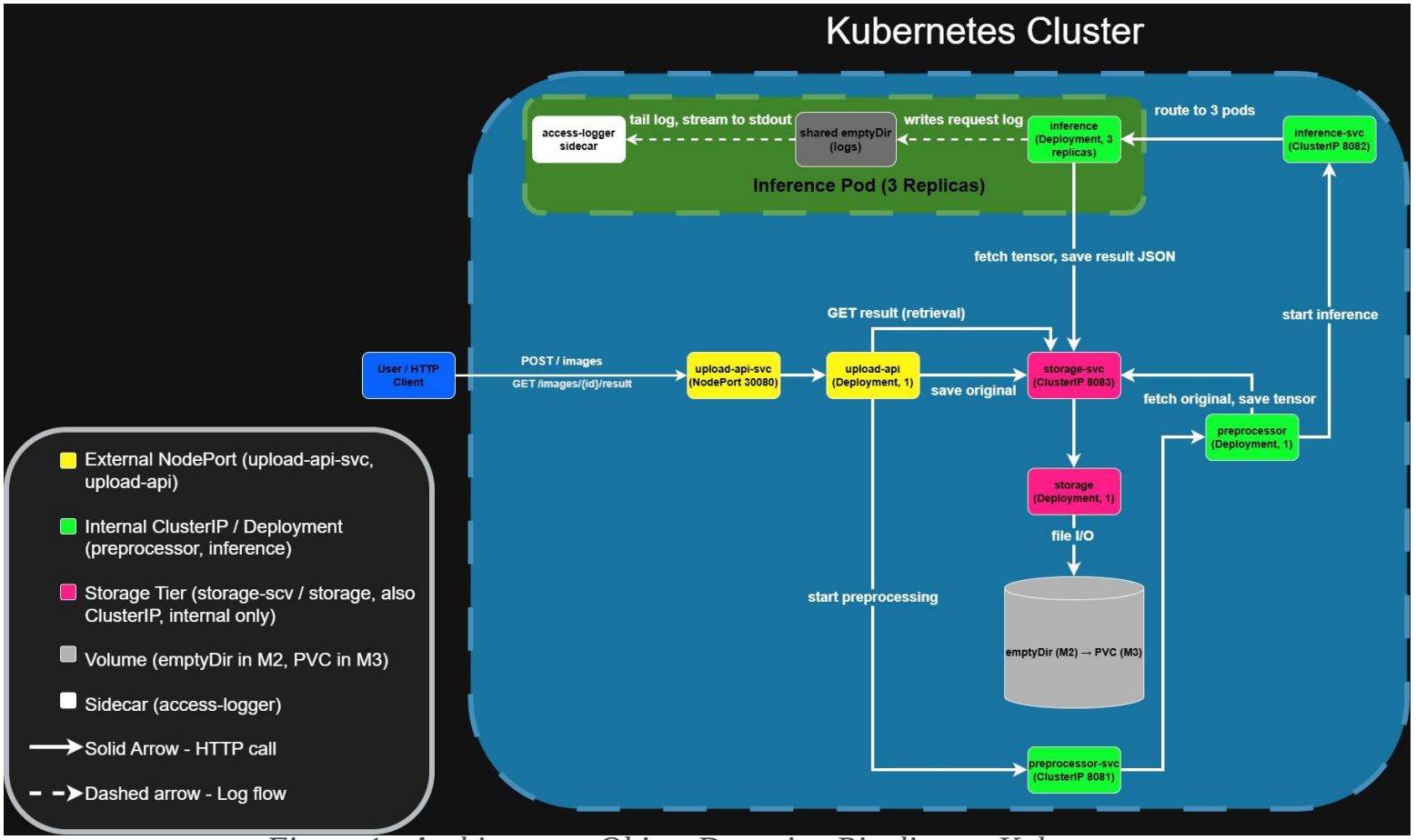


Figure 1 - Architecture: Object Detection Pipeline on Kubernetes

The architecture strictly enforces project constraints and cloud-native design patterns. In accordance with requirements, the system exposes exactly one external entry point: the upload-api-svc Service bound via NodePort, which shields internal pipeline components from unauthorized direct access. All inter-service communications occur entirely over private Kubernetes ClusterIP Services using internal CoreDNS names, making intermediate stages completely invisible from outside the cluster. The inference Deployment runs three replicas in Milestone 2 because it is compute-intensive and entirely stateless, which allows replica scaling without distributed coordination; Horizontal Pod Autoscaler (HPA) based scaling on CPU utilization is deferred to a later milestone. A co-located access-logger sidecar container tails the inference container's request log from a shared emptyDir volume and streams it to stdout, demonstrating the multi-container Pod pattern. Only the storage Deployment accesses the data volume. In Milestone 2 this is an emptyDir volume organized into /data/originals/, /data/processed/, and /data/results/, where UUID-based naming prevents write collisions under concurrent uploads; a PersistentVolumeClaim replaces it in Milestone 3. Finally, pinned container image tags ensure clean reprovisioning via kubectl delete -f deployments/ -f services/&& kubectl apply -f deployments/ -f services/ with zero configuration drift.

Chapter 2: Implementation Proposal

2.1 Use Cases & User Stories

Story 1 - Upload & Receive ID: As a user, I upload a JPEG image via a REST API and immediately receive a UUID confirmation, so that I can retrieve my object detection result asynchronously without blocking my application.

Story 2 - Retrieve Detection Results: As a user, I query the API with my UUID and receive a structured list of every detected object label, confidence score, and bounding box coordinates so that I can overlay annotations on the original image or feed results into a downstream system.

Story 3 - Operator Scale-Out: As a system operator, I horizontally scale the inference Deployment during peak load periods, so that detection throughput increases without redeploying or reconfiguring any other service in the pipeline.

2.2 Data-Flow Diagram

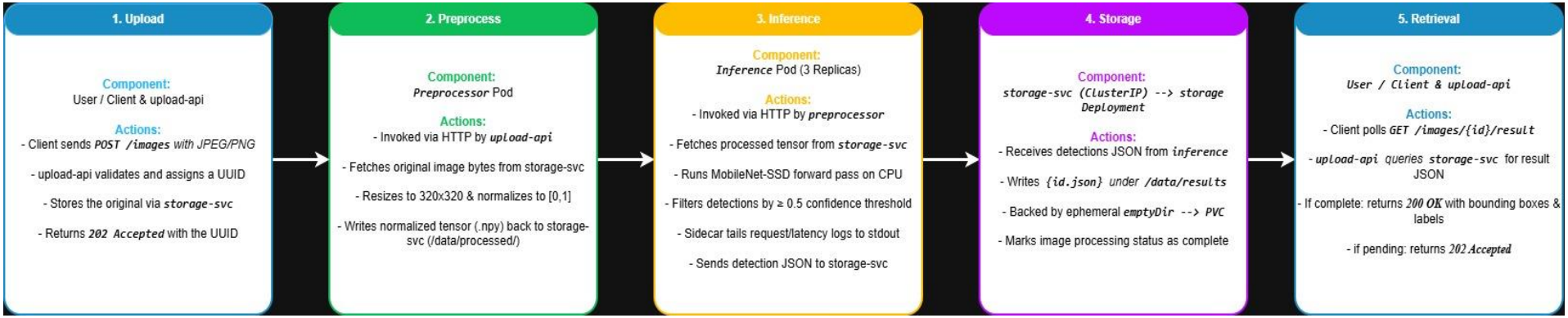


Figure 2 - Data-Flow Diagram: upload → preprocessing → inference → storage → retrieval

The data pipeline operates entirely asynchronously to ensure fast user response times and robust fault tolerance. When a client submits an image at Step 1, upload-api validates payload constraints, assigns a UUIDv4, forwards the raw file to storage-svc, and returns an immediate 202 Accepted containing the tracking UUID.

The call chain behind the scenes proceeds as follows:

upload-api invokes preprocessor-svc with the UUID.

preprocessor fetches the original raw image bytes from storage-svc, resizes and normalizes the image, writes the resulting .npy tensor back to storage-svc (/data/processed/), and invokes inference-svc.

inference fetches the processed tensor from storage-svc, executes the MobileNet-SSD forward pass on CPU, filters detections by confidence score (≥ 0.5

≥0.5), and posts the structured detection JSON back to storage-svc (/data/results/).
An access-logger sidecar running inside each inference Pod tails the local access logs from a shared volume and pipes them to stdout.
In Step 5, the client polls upload-api (GET /images/{id}/result), which queries storage-svc for the final JSON payload and returns 200 OK (or 202 Accepted if processing is still underway).

2.3 Kubernetes Component Design

The pipeline is decomposed into four Deployments plus one sidecar container, each with a single responsibility.

Table 1: Kubernetes Pipeline Workloads

Component	Kind	Replicas	Pinned Image Tag	Role
upload-api	Deployment	1	upload-api:1.0.0	External REST API; UUID assignment; result retrieval
preprocessor	Deployment	1	preprocessor:1.0.0	Resize to 320×320, normalize to [0,1]
inference	Deployment	3	inference:1.0.0	MobileNet-SSD detection forward pass
storage	Deployment	1	storage-svc:1.0.0	CRUD abstraction layer over ephemeral storage (PVC in M3)
access-logger sidecar	Sidecar (in inference Pod)	N/A	busybox:1.36	Tails the shared request log and streams it to stdout

upload-api: External FastAPI gateway; validates JPEG/PNG (≤ 2 MB), assigns UUIDv4, streams raw files to storage-svc, dispatches preprocessing, and serves client result polling.

preprocessor: Input transformation service; fetches the raw asset from storage-svc by UUID, resizes to 320×320, normalizes pixel values to [0,1], and saves the binary NumPy tensor (.npy) via storage-svc to /data/processed/.

inference: Computational core with pinned PyTorch/torchvision; runs MobileNet-SSD forward pass on CPU, applies a 0.5 confidence threshold, and emits structured detection JSON to storage-svc.

storage-svc: Internal CRUD REST API over ephemeral storage (PVC in Milestone 3); manages file I/O, prevents lock collisions, and isolates the storage backend from compute pods.

Multi-container Pod note: The inference Pod is the only multi-container Pod in the pipeline, co-locating MobileNet-SSD with an access-logger sidecar. *Sidecar Rationale:* The inference container writes request and latency logs to a shared emptyDir volume. The sidecar tails that file and streams it directly to stdout, allowing logs to be collected using standard Kubernetes tooling (kubectl logs) without modifying the core inference application code. This decouples auxiliary operational logging from the primary deep learning workload.

2.4 Service Choices & Scaling Target

Table 2: Kubernetes Service Configurations

Service	Type	Port → targetPort	NodePort	Justification
upload-api-svc	NodePort	80 → 8080	30080	Single external entrypoint which satisfies the one-entrypoint constraint
preprocessor-svc	ClusterIP	8081 → 8081	N/A	Internal pipeline stage only
inference-svc	ClusterIP	8082 → 8082	N/A	Internal pipeline stage only
storage-svc	ClusterIP	8083 → 8083	N/A	Internal I/O abstraction but not externally reachable

The inference Deployment is designated as the horizontal scaling target. it is manually scaled to 3 replicas for high availability and throughput:

kubectl scale deployment inference --replicas=3

In a future milestone, dynamic scaling will be governed by a HorizontalPodAutoscaler (HPA) configured as follows:

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: inference-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: inference
  minReplicas: 3
  maxReplicas: 5
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization:70
```

Scaling Rationale: The inference Deployment is stateless and compute-heavy, making it the primary throughput bottleneck in the pipeline. Running three replicas ensures resilience and concurrent processing capacity. All other Deployments remain at 1 replica because their lightweight asynchronous I/O and preprocessing workloads do not bottleneck under ≤ 2 MB inputs. Automated HPA scaling is planned for a later milestone and requires metrics-server and CPU resource requests to be active in the cluster.

2.5 Persistence Plan

One PVC: pipeline-pvc accessModes: ReadWriteOnce, storage: 5Gi, hostPath for local development. Directory layout:

```
/data/
  originals/{uuid}.jpg     <- raw user upload (JPEG/PNG <= 2 MB)
  processed/{uuid}.npy     <- normalized 320x320 float32 tensor
  results/{uuid}.json      <- { detections: [{label, confidence, bbox}] }
```

Table 3: Pipeline Artifact Persistence Strategy

Artifact	Format	Written By	Read By	Directory
Original image	.jpg / .png	Storage (on behalf of upload-api)	preprocessor (via storage)	/data/originals/
Normalized tensor	.npy	storage (on behalf of preprocessor)	Inference (via storage)	/data/processed/
Detection result	.json	storage (on behalf of inference)	upload-api (retrieval, via storage)	/data/results/

UUID filenames prevent write collisions during concurrent uploads. The storage Deployment is the sole component with access to the data volume; all others call it via the storage-svc ClusterIP Service to ensure unified file management.

2.6 API Contracts

POST /images: Upload an image

```
Content-Type: multipart/form-data
Body field:   file = <image bytes>  (JPEG or PNG, max 2 MB)

202 Accepted:
{ "id": "550e8400-e29b-41d4-a716-446655440000", "status": "processing" }

413 Payload Too Large:  image exceeds 2 MB limit
415 Unsupported Media Type:  file is not JPEG or PNG
```

GET /images/{id}/result: Retrieve detection result

```
200 OK:
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "timestamp": "2026-09-25T14:32:00Z",
  "detections": [
    { "label": "person",  "confidence": 0.94, "bbox": [120, 45, 80, 210] },
    { "label": "bicycle", "confidence": 0.76, "bbox": [300, 90, 150, 180] }
  ]
}

202 Accepted:  { "status": "processing" }  <- inference not yet complete
404 Not Found:  { "error": "not found" }
```

2.7 Risk & Test Plan

Top Risks

Table 4: Pipeline Risks and Mitigations

1	Risk	Likelihood	Mitigation
1	Model checkpoint (~20 MB) slows Pod startup	Medium	Weights baked into the image (or pre-pulled by an initContainer); readinessProbe delays traffic until ready
2	Write contention under concurrent uploads	Medium	UUID filenames eliminate collisions; storage serializes writes if needed
3	Large images exhaust memory or exceed timeout	High	2 MB limit enforced at API layer; resources. limits set on inference Pod
4	Data lost on Pod restart	Medium	Accepted for Milestone 2; PVC planned for Milestone 3. Documented teardown script included in repo
5	PyTorch/torchvision version drift	Low	All image tags and requirements.txt pinned to exact versions
6	Low-confidence detections cluttering results	Medium	Configurable confidence threshold (default 0.5) applied at inference time

Test Matrix

Table 5: System Verification Test Matrix

Test Case	Input	Expected Output	Method
Happy path	Valid JPEG, 100 KB, visible objects	HTTP 202 → poll → 200 with detections array	curl + kubectl port-forward
Empty scene	Plain white JPEG	HTTP 200, "detections": []	curl
Large image	PNG, 5 MB	HTTP 413	curl
Invalid format	PDF file	HTTP 415	curl
10 concurrent uploads	10 valid JPEGs	10 unique IDs, all eventually 200	Parallel curl loop
Clean reprovision	kubectl delete -f . && kubectl apply -f .	All Pods reach Running; no residual errors	kubectl get pods
Self-healing	kubectl delete pod	A new Pod is created automatically; inference count returns to 3	kubectl get pods -w
Manual scale	kubectl scale deployment inference --replicas=4	4 inference Pods Running	kubectl get deploy,pods
HPA scale-out	Sustained CPU load via concurrent requests	New inference replica observed	kubectl get hpa

Logan Yates

Email: loganllyates@yahoo.com
LinkedIn: https://www.linkedin.com/in/logan-yates-50771b256/ | GitHub: https://github.com/LegionYata

EDUCATION

West Chester University — **West Chester, PA**
Bachelor's Degree in Computer Science (Cloud Engineering) & Minor in Music Production — August 2022 - May 2026
GPA: 3.5

Honors:
Cum Laude
Dean's Lists (Fall 2022, Spring 2023,Spring 2026) Honor Roll

Scholarships:
The Woodman McIntosh STEM Opportunity Scholarship Fund
The College of the Sciences and Mathematics (CSM) Scholarship

Relevant Coursework:
Computer Science III, Data Structures and Algorithms, Modern Web Applications, Operating Systems, Modern Web Applications, Database Management Systems, Data Communications & Networking, Data Analytics, Software Engineering, Intro. to Cloud Computing.

CAMPUS INVOLVEMENT

Competitive Programming Club
Member — September 2025 – May 2026

- Participated in monthly programming competitions with club members to solve difficult coding problems.
- Discussed strategies to carry through coding problems much more efficiently.
- Conducted retrospective analysis of archived problems to refine Dynamic Programming and optimize Graph Algorithms.

Computer Science Club
Member — August 2022 – May 2026

- Engaged in monthly Lunch & Learn talks hosted by different software companies.
- Participated in creating robots from scratch by having them view and observe human motion.
- Corrected weak code during mock interviews to demonstrate an understanding of algorithmic principles.

New Student Orientation
Orientation Leader May 2025 – July 2025

- Guided groups of 20+ students and families by being a point of contact.
- Provided information about the campus resources and tools for the students to utilize.
- Collaborated with other orientation leaders to help improve overall experiences for all.

Dowdy Multicultural Center - Peer Mentoring Program
Peer Mentor August 2024 – May 2026

- Supported students of color to successfully integrate academically and socially into university life.
- Attended other multicultural organization events alongside mentee(s).
- Done weekly meetups for checking on well-being and providing recommended personal resources.

Brothers Of Excellence
Board Member September 2025 – May 2026

- Connected with industry professionals and alumni to foster networking opportunities.
- Coordinated volunteer efforts for other conferences in partnership with local non-profit organizations.
- Allocate professional development resources and workshops for students to build leadership.

EXPERIENCE

Graduate Assistant
Research Assistant August 2026 – Present

- Contributed to the Audiovisual Labeled Emotion (ALE) Research Platform, a low-cost system for collecting labeled audiovisual data on human behavior and interaction.
- Collected and organized self-reported affect and activitydata in natural educational and human-interaction settings.
- Annotated audiovisual recordings by labeling observed emotions, activities, and interactions to support the development of behavioral research datasets.